Claude Code 워크북

미래에셋생명 · AI 코딩 어시스턴트 활용 워크숍

5시간 과정 가이드북

목차

Part 1. 도입 5

1.1 LLM이 바꾼 업무 방식 5

1.2 챗봇과 에이전트는 어떻게 다른가요 5

1.3 실제 사례 — 다른 회사들이 얻은 효과 6

1.4 오늘의 목표 7

1.5 워크숍 환경 안내 8

1.6 오늘 배울 도구 한눈에 9

Part 2. 개념 설명 및 시연 11

2.1 Claude Code 시작하기 11

2.2 좋은 프롬프트를 쓰는 법 12

2.3 Plan 모드 — 본격 작업 전에 설계부터 12

2.4 권한 시스템 — 매번 Yes를 안 누르는 법 13

2.5 CLAUDE.md — 프로젝트 규칙서 14

2.6 Skill — 재사용 가능한 업무 매뉴얼 15

2.7 Command — 단축 명령어 16

2.8 Hook — 자동 안전장치 17

2.9 컨텍스트 관리 18

2.10 도구의 병렬 실행 19

2.11 성공 패턴 — 빠른 반복이 완벽을 이깁니다 19

2.12 바이브코딩 주의사항 20

Part 3. 기능 체험 21

미션 1. Plan 모드로 기능 정의하기 21

미션 2. 권한 설정하기 21

미션 3. CLAUDE.md 만들기 22

미션 4. Skill 만들기 22

미션 5. Command 만들기 22

미션 6. Hook 만들기 22

미션 7. 컨텍스트 관리 23

미션 8. 노트 테이킹 23

Part 4. 실습 프로젝트 25

4.1 7단계 워크플로우 25

4.2 완성하면 무엇이 손에 남을까요 25

Appendix A. 워크숍 이후 — 집에서 혼자 쓰기 27

A.1 Claude Code (CLI) 개인 PC 설치 27

A.2 Claude Desktop (GUI 앱) 설치 28

A.3 CLI vs Desktop 비교 29

A.4 실행 흐름과 자주 쓰는 명령 29

A.5 안전하게 쓰기 위한 주의사항 30

Part 1. 도입

1.1 LLM이 바꾼 업무 방식

예전에는 AI에게 무언가를 시키려면 코드를 직접 짜야 했습니다. 또는 사람이 손으로 모든 것을 처리했습니다. 보고서 한 편을 만들려면 자료 조사부터 분류, 본문 작성, 검수까지 며칠이 걸렸습니다.

지금은 자연어로 말하면 AI가 알아서 코드를 만들고 실행합니다. 보고서, 분석, 자동화 같은 결과물이 곧장 손에 들어옵니다. 사람은 "무엇을, 왜, 어떻게" 만 결정하면 됩니다.

이것을 가능하게 하는 도구가 **AI 코딩 어시스턴트**입니다. 그중에서도 우리가 오늘 사용할 도구는 **Claude Code** — Anthropic의 Claude 모델을 터미널에서 직접 부릴 수 있게 만든 공식 CLI입니다.

Claude Code의 본질은 단순합니다. "사람이 채팅창에 자연어로 지시하면, Claude가 사람을 대신해서 실제 컴퓨터를 다룹니다." 이게 전부입니다. 하지만 이 단순한 본질로부터 업무 자동화의 풍경이 바뀝니다. 한 번에 한 줄 지시만으로 자료 조사, 본문 작성, 표 생성, 차트 그리기, 파일 저장까지 자동으로 흘러갑니다.

1.2 챗봇과 에이전트는 어떻게 다른가요

AI를 활용하는 방식에는 두 가지가 있습니다. 하나는 익숙한 **챗봇**이고, 다른 하나는 오늘 우리가 다룰 **에이전트**입니다.

구분	챗봇 (ChatGPT 무료 등)	에이전트 (Claude Code)
입력	텍스트 질문	텍스트 지시
출력	텍스트 답변	파일 생성 · 코드 실행 · 결과물
컴퓨터 접근	없음	직접 파일 · 터미널 사용
다단계 작업	사람이 단계마다 입력	알아서 끝까지
결과물 종류	텍스트 한 덩어리	보고서, 코드, 데이터, 이미지 등

핵심 차이는 단순합니다. 챗봇은 "말로 답합니다", 에이전트는 "행동합니다". 챗봇과 대화할 때는 결과물을 사람이 옮겨 적어야 하지만, 에이전트는 결과물을 파일로 직접 만들어 둡니다.

비유하자면 챗봇이 카카오톡 상담사라면, 에이전트는 신입 인턴입니다. 인턴에게 "분기 보고서 만들어 주세요" 라고 하면, 인턴은 자료를 찾고, 본문을 쓰고, 파일을 폴더에 정리해서 다시 가져옵니다. 우리가 오늘 만들 것이 바로 이 "잘 훈련된 인턴"입니다.

같은 질문, 다른 답변

같은 질문을 두 도구에 던져 보면 차이를 한눈에 알 수 있습니다.

질문: "퇴직연금 시장 분석 보고서를 만들어 주세요."

일반 챗봇의 답변

> 퇴직연금 시장 분석 보고서를 작성하시려면 다음 항목을 고려해 보세요:
>
> 1. 시장 규모 및 성장률
> 2. 주요 사업자별 점유율
> 3. 상품 트렌드 (DC/DB/IRP)
> 4. 규제 변화
> 5. 향후 전망
>
> 필요하시면 보고서 템플릿을 만들어 드릴 수 있습니다. 어떤 형식을 원하시나요?

이 답변을 받아도 사람이 다시 자료를 찾고, 워드에 옮겨 적고, 차트를 만들어야 합니다. 보통 반나절에서 하루가 걸립니다.

Claude Code(에이전트)의 답변

```
● Read CLAUDE.md (3.2k)
● Skill ai-plan-report 적용
● WebFetch 금감원 통계 (2026-Q1)
● WebFetch 보험연구원 보고서
● Write outputs/퇴직연금_분석.md
● Bash python make_chart.py
● Write outputs/퇴직연금_분석.docx
✓ 완료. 12페이지 .docx 생성
  - 시장 규모 18.9조 원 (+7.2%)
  - 차트 4종 포함
  - 출처 각주 12개
```

`outputs/` 폴더에 완성된 파일이 들어가 있습니다. 약 3분이 걸렸습니다. 우리는 그 파일을 열어 보고, 마음에 안 드는 부분만 수정 요청을 다시 던지면 됩니다.

1.3 실제 사례 — 다른 회사들이 얻은 효과

Claude Code가 단순한 호기심 거리가 아니라 실제 업무에 영향을 주고 있다는 것을, 공식적으로 발표된 사례들을 통해 확인해 보겠습니다. 출처는 모두 Anthropic 공식 사이트, CIO 매거진, Medium 등의 공개 문서입니다.

조직 / 사례	효과	내용
Anthropic 마케팅팀	2.5시간 → 30분	고객 사례 보고서 작성 시간 단축. 주당 10시간 절약.
ServiceNow (직원 29,000명)	준비 시간 95% 단축	영업 미팅 준비 자료를 Claude Code가 자동 생성.
Block (결제 기업)	운영 효율 40% 향상	결제·상점 데이터를 Claude로 실시간 문서화.
DevOps 엔지니어 사례	응답 시간 5초 → 0.5초	Redis CPU 급증 원인을 5분 만에 분석, 10배 개선.
비개발자 마케터 (Austin Lau)	30분 → 30초	Figma 광고 시안 생성 플러그인을 코드 없이 제작.
엔터프라이즈 평균	엔지니어 생산성 30~60%	코드 리뷰·디버깅·문서화 자동화로 PR 처리 30% 빠름.

시사점

- AI 도입은 더 이상 "코딩 잘하는 사람만" 의 영역이 아닙니다. 비개발자도 한 줄 지시로 실제 결과물을 만들고 있습니다.
- 효과는 크게 두 가지로 갈립니다. 하나는 **반복 업무 시간 단축** (작성·정리·문서화), 다른 하나는 **수동으로는 못 하던 일의 자동화** (실시간 데이터 수집, 차트 자동 생성 등).
- 오늘 워크숍의 목표는 여러분도 같은 효과를 자기 부서에 가져갈 수 있게 하는 것입니다.

1.4 오늘의 목표

5시간 뒤, 여러분은 다음 네 가지가 가능해집니다.

1. Claude Code를 켜서 자연어 한 줄로 보고서를 만들 수 있습니다.
2. 본인 부서의 문서 자동화 프로그램을 직접 설계할 수 있습니다.
3. CLAUDE.md, Skill, Command, Hook 등 핵심 기능 다섯 가지를 활용할 수 있습니다.
4. 다음에 새로운 자동화가 필요할 때 혼자서도 같은 패턴으로 만들 수 있습니다.

이 워크숍은 Claude Code의 모든 기능을 가르치는 것이 목표가 아닙니다. **자기 일에 적용할 수 있는 최소 핵심 다섯 가지 + 한 개의 완성된 결과물**, 이 두 가지를 들고 가시는 것이 목표입니다.

1.5 워크숍 환경 안내

오늘은 본인 PC에 아무것도 설치하지 않습니다. 브라우저로 워크숍 서버에 접속하시면 모든 것이 한 화면에 들어 있습니다.

공통 컨트롤 (모든 슬라이드)

상단 툴바에서 다음을 조작할 수 있습니다.

버튼	기능
⛶	전체화면 (F5 키도 가능)
☀ / ⛶	라이트 / 다크 모드 전환
⛶ / +	화면 확대·축소 (7단계)
← →	이전 / 다음 슬라이드 (키보드 화살표도 가능)
⛶강의 모드	강사가 진행을 통제할 때 표시 — 슬라이드가 자동으로 넘어갑니다

미션 슬라이드 구성

체험/실습 미션에서는 화면이 두 칸으로 나뉩니다.

- **왼쪽 (35%)** — 안내 패널
- ⛶**GOAL** — 이 미션의 목적
- ⛶**INPUT** — 내가 무엇을 입력해야 하는지
- ⛶**OUTPUT** — 어떤 파일이 나와야 하는지 (파일 칩이 생성되면 회색→녹색으로 바뀜)
- ⛶프롬프트 + 힌트 + 모범답안
- ⛶필수 체크리스트 + ⛶도전 체크리스트
- ⛶**AI 채점 받기** 버튼 — 산출물을 AI가 상세 평가 (파일 생성 후 활성화)
- **오른쪽 (65%)** — 터미널 (Claude Code가 이미 설치되어 있고 실행 권한도 준비됨)
- 우측 하단에 ⛶**AI 조수** 카드가 자동으로 떠서 진행 상황을 코칭합니다

미션의 산출물 파일이 만들어지면 ⛶**토스트**(큰 알림)가 터미널 위에 뜨고, 체크리스트가 자동 통과되어 미션 클리어됩니다.

환경

각 사용자는 격리된 자기 공간(`userXX`)을 받습니다. 옆 사람과 파일이 섞이지 않고, 실수해도 다른 사람에게 영향을 주지 않습니다. 강사(`user00`)의 진행에 맞춰 자동으로 같은 슬라이드로 이동합니다.

오늘 환경에서 사용할 작업 폴더는 `subagent-mastery` 입니다. 이 폴더는 미리 준비되어 있고, 터미널은 자동으로 이 폴더에서 시작합니다. 따로 `cd` 명령을 칠 필요가 없습니다.

1.6 오늘 배울 도구 한눈에

Part 2와 Part 3에서 다룰 핵심 여섯 가지

- **Plan 모드** — 본격 작업 전에 단계별 설계를 받아 검토합니다.
- **권한 시스템** — 매번 Yes를 누르지 않게 자동 허용 목록을 등록합니다.
- **CLAUDE.md** — 매 대화 시작 시 자동으로 적용되는 프로젝트 규칙서입니다.
- **Skill** — 재사용 가능한 표준 업무 매뉴얼입니다 (예: 보고서 작성 절차).
- **Command** — `/report` 처럼 한 단어로 호출되는 단축 명령어입니다.
- **Hook** — 특정 상황에서 자동 실행되는 안전장치입니다 (예: 개인정보 차단).

Part 4 실습 프로젝트 — 위 도구를 모두 활용해서 본인 업무에 맞는 자동화 한 개를 처음부터 끝까지 완성하고, 마지막엔 다른 사람도 쓸 수 있는 웹페이지로 확장합니다.

Part 2. 개념 설명 및 시연

2.1 Claude Code 시작하기

Claude Code = 나만의 AI 비서

Claude Code는 터미널 안에서 동작하는 Claude입니다. 채팅창처럼 자연어로 말하면 Claude가 다음과 같은 일을 합니다.

1. 폴더를 살펴보고 파일을 읽습니다.
2. 새 파일을 만들거나 기존 파일을 수정합니다.
3. 파이썬·노드 같은 코드를 직접 실행해서 결과를 봅니다.
4. 인터넷을 검색해 최신 정보를 가져옵니다.
5. 위 과정을 자율적으로 여러 단계 이어서 실행합니다.

다시 말해, Claude Code는 "사람을 대신해 컴퓨터를 다루는 잘 훈련된 인턴"입니다.

첫 실행

워크숍 환경에서는 브라우저로 본인 ID(`userXX`)에 접속하면 터미널이 이미 열려 있습니다. 거기서 한 줄을 입력합니다.

```
claude
```

이게 끝입니다. Claude Code가 시작되고 입력 프롬프트(`>`)가 나타납니다. 이제 자연어로 무엇이든 지시할 수 있습니다.

첫 명령

가벼운 인사부터 시작해 보세요.

```
> 안녕! 자기소개 해줘
```

또는 바로 본격적인 일을 시킵니다.

```
> 퇴직연금 분기 보고서 자동 생성 프로그램의 구조를 잡아줘
```

처음에는 Claude가 이것저것 묻거나 권한을 확인합니다. 이 워크숍에서는 그냥 Yes를 누르시면 됩니다 (격리된 환경이라 안전합니다). 익숙해지면 권한을 미리 등록해서 묻지 않게 만들 것입니다 (2.4 절).

2.2 좋은 프롬프트를 쓰는 법

좋은 프롬프트의 4요소

좋은 프롬프트는 보통 네 가지 요소를 모두 포함합니다.

1. **주제** — 무엇에 대한 건지
2. **목적** — 왜 만드는지, 어디에 쓸 건지
3. **형식** — 어떻게 나와야 하는지 (.docx? 표? 다섯 단락?)
4. **제약** — 하지 말아야 할 것 (영어 금지, 특정 폴더에만 저장, 250단어 이내 등)

나쁜 프롬프트와 좋은 프롬프트

나쁜 예부터 보겠습니다.

> 보고서 만들어줘

이렇게 말하면 Claude는 "어떤 보고서요?", "어떤 형식으로요?" 라고 되물을 수밖에 없습니다.

좋은 예는 이렇습니다.

> 2025년 퇴직연금 시장 현황을 금감원과 보험연구원 데이터로 조사하고, 핵심 수치와 주요 트렌드를 정리해서 Word 파일(.docx)로 만들어 주세요. 제목, 목차, 본문, 결론 포함. outputs/ 폴더에 저장. 한국어로 작성.

좋은 프롬프트는 주제, 목적, 형식, 저장 위치, 언어까지 모두 명확합니다. AI가 추측할 여지가 없으니 한 번에 의도한 결과가 나옵니다. 그리고 이 4요소 중 자주 반복되는 것들은 CLAUDE.md에 한 번만 적어두면 매번 안 써도 됩니다 — 이게 2.5 절의 핵심입니다.

2.3 Plan 모드 — 본격 작업 전에 설계부터

왜 먼저 설계할까요

복잡한 작업을 바로 실행하면 엉뚱한 결과가 나오기 쉽습니다. AI는 의도를 추측해서 빠르게 결과물을 내는데, 그 추측이 사용자가 원한 것과 다를 수 있기 때문입니다. 그래서 본격 실행 전에 **단계별 계획**을 먼저 받아 검토하면 안정성이 크게 올라갑니다.

사용법

```
/plan AI 추진 TF의 "AI 추진 계획 보고서"를 자동으로 만드는
프로그램을 설계할 거야. 단계별 계획을 세워줘.
```

`/plan` 으로 시작하면 Claude는 작업을 즉시 실행하지 않고, 4-5단계의 계획을 먼저 제시합니다. 사용자는 이 계획을 검토한 뒤 다음과 같이 응답할 수 있습니다.

- "좋아요. 그대로 실행해 주세요."
- "2번 단계는 빼고 진행해 주세요."
- "사례 리서치 단계를 두 단계로 나눠 주세요."

효과

- **방향 조기 차단**: 의도와 다른 방향이면 실행 전에 막을 수 있습니다.
- **비용 예측**: 실행에 얼마나 걸릴지, 어떤 도구가 호출될지 미리 알 수 있습니다.
- **사용자가 흐름을 통제**: AI가 자율 실행하는 것이 아니라, 사람이 검토 후 승인하는 형태입니다.

Plan 모드는 처음 작업할 때나 익숙하지 않은 영역을 다룰 때 특히 유용합니다. 익숙해지면 단순한 작업은 Plan 없이 바로 실행해도 됩니다.

2.4 권한 시스템 — 매번 Yes를 안 누르는 법

왜 자꾸 Yes를 눌러야 할까요

Claude Code는 안전을 위해 파일 쓰기, 명령 실행 등 위험할 수 있는 작업을 매번 사용자에게 확인합니다. 처음 사용할 때는 "Yes"를 수십 번 눌러야 할 수도 있습니다. 이것은 의도된 동작이고, 사용자가 모르는 사이에 시스템에 변화가 일어나지 않도록 막는 안전장치입니다.

한 번에 권한 설정하기

매번 Yes 누르는 게 번거롭다면 `.claude/settings.local.json` 파일에 미리 허용 목록을 등록합니다.

```
{
  "permissions": {
    "allow": [
      "Bash(*)",
      "Read(*)",
      "Write(*)",
      "Edit(*)"
    ]
  }
}
```

이제부터 Bash, Read, Write, Edit 도구는 자동 허용됩니다. 매번 묻지 않습니다. 더 좁게 설정하고 싶으면 `Bash(python *)` 처럼 특정 명령만 허용하는 형태도 가능합니다.

무조건 Yes 눌러도 괜찮을까요

워크숍 환경에서는 괜찮습니다. 격리된 사용자 홈에서만 동작하고, 강사가 언제든 초기화할 수 있습니다. 마음 편히 Yes를 누르셔도 됩니다.

실제 업무 환경에서는 신중해야 합니다. 본인 PC나 회사 시스템에 적용할 때는 다음을 권장합니다.

- `Bash(*)` 같은 와일드카드 대신 `Bash(python *)`, `Bash(npm *)` 등 좁은 패턴
- 파일 쓰기는 특정 폴더로만 제한
- 위험한 명령(`rm -rf`, `git push --force` 등)은 절대 자동 허용하지 않기

2.5 CLAUDE.md — 프로젝트 규칙서

무엇인가요

프로젝트 폴더에 `CLAUDE.md` 파일을 두면 Claude Code가 매 대화 시작 시 자동으로 그 내용을 읽습니다. 일종의 "이 프로젝트에서 일할 때의 작업 규칙서"입니다. 한 번 적어두면 매 프롬프트마다 반복할 필요가 없어집니다.

구조 예시

```
# 프로젝트
- 팀: AI 추진 TF
- 목적: AI 추진 계획 보고서 자동 생성

## 코딩 규칙
- 모든 응답, 주석, 문서는 한국어
- 강조 색상: #F58220 (브랜드 오렌지)
- 모든 보고서는 outputs/ 폴더에 저장

## 주요 작업
- 리서치 → 본문 작성 → 검토 → .docx 출력
```

효과: 전후 비교

CLAUDE.md가 없을 때

> 사용자: "보고서 만들어 주세요"
>
> Claude: 영어로 답하거나, 어디에 저장할지 모르거나, 일반적인 톤으로 작성

CLAUDE.md가 있을 때

> 사용자: "보고서 만들어 주세요"
>
> Claude: 한국어 + outputs/ 폴더 + 우리 팀 톤 + 브랜드 색상 적용

같은 한 줄 명령이지만 결과의 품질이 완전히 달라집니다. CLAUDE.md는 "AI를 우리 팀에 입사시키는 신입사원 교육 매뉴얼"이라고 생각하시면 됩니다.

2.6 Skill — 재사용 가능한 업무 매뉴얼

무엇인가요

특정 업무의 표준 절차를 `.claude/skills/<이름>/SKILL.md` 파일에 정리해 두는 것입니다. Claude가 그 업무를 수행할 때 자동으로 SKILL.md를 따라 일합니다.

CLAUDE.md가 "프로젝트 전반의 규칙"이라면, Skill은 "특정 작업의 표준 절차서"입니다. 한 프로젝트 안에 Skill은 여러 개 존재할 수 있습니다.

구조 예시

```
---
name: ai-plan-report
description: AI 추진 계획 보고서 작성 표준 절차
---

## 입력
- 부서명
- 프로젝트 목표

## 절차
1. 추진 배경 정리 (3~5문장)
2. 적용 사례 리서치 (3개 이상)
3. 도입 로드맵 작성 (단계별 일정)
4. 위험·기대효과 정리 (각 3개)
5. 결론 + .docx 파일로 출력

## 출력
- outputs/ai-plan-{부서명}-{날짜}.docx
```

효과

- **한 줄 호출**: "AI 추진 보고서 만들어 줘" 한 줄이면 위 5단계가 자동 실행됩니다.
- **품질 일관성**: 누가 시키든 동일한 구조의 결과물이 나옵니다.
- **팀 공유**: Skill 파일을 깃에 올리면 팀 전체가 같은 절차로 일하게 됩니다.

옆 사람과 동시에 같은 Skill을 호출해 보면, 거의 같은 구조의 보고서가 나오는 것을 확인할 수 있습니다. 이게 Skill의 위력입니다.

여러 예시

- `report-writer` — 일반 보고서 작성
- `competitor-watch` — 경쟁사 4곳 모니터링 + 비교표
- `compliance-check` — 작성된 문서의 규제 위반 여부 검토
- `meeting-summary` — 회의록 요약 + 액션 아이템 추출

2.7 Command — 단축 명령어

무엇인가요

`/report`, `/full-analysis` 같은 슬래시 명령어를 직접 만들 수 있습니다. `.claude/commands/<이름>.md` 파일이 곧 명령어 정의입니다. 명령어는 여러 Skill을 순서대로 호출하는 워크플로우 역할을 합니다.

구조 예시

```
---
description: 주제를 입력하면 보고서, PPT, 컴플라이언스 검토까지 자동 실행
argument-hint: [주제 - 예: 퇴직연금]
allowed-tools: Read, Write, Bash, WebFetch
---

다음 단계를 순서대로 실행:

1. ai-plan-report 스킬을 사용하여 "$ARGUMENTS" 주제로 보고서 작성
2. 같은 템플릿으로 PowerPoint 생성
3. compliance-check 스킬로 규제 검토
4. outputs/ 폴더에 모든 결과물 저장
```

사용법

```
/report 2025 퇴직연금 시장 동향
```

위 4단계가 한 번에 실행됩니다. 매번 긴 프롬프트를 입력하지 않아도 단어 하나로 워크플로우 전체가 호출됩니다.

효과

- **반복 작업 단축**: 자주 쓰는 워크플로우를 한 단어로 호출합니다.

- **공유 가능**: `.claude/commands/` 폴더를 깃에 올리면 팀이 같은 명령어를 사용합니다.
- **인자 지원**: `$ARGUMENTS` 로 주제, 부서명 등 동적 입력을 받습니다.

2.8 Hook — 자동 안전장치

무엇인가요

특정 도구가 호출되기 직전 또는 직후에 자동으로 실행되는 스크립트입니다. 예를 들어 "파일 쓰기 직전에 개인정보 검사", "코드 작성 후 자동 포맷팅" 같은 안전장치를 만들 수 있습니다.

Hook의 핵심은 **사용자가 명시한 안전장치는 절대 우회되지 않는다**는 점입니다. Claude가 아무리 똑똑해도, Hook이 막으면 그 동작은 실행되지 않습니다.

예시: 개인정보 차단 Hook

```
{
 "hooks": {
  "PreToolUse": [
   {
    "matcher": "Write",
    "hooks": [{
     "type": "command",
     "command": "bash .claude/hooks/check-pii.sh"
    }]
   }
  ]
 }
}
```

`check-pii.sh` 가 파일 내용에서 주민번호 패턴(`\d{6}-\d{7}`)을 찾으면 0이 아닌 종료 코드로 끝냅니다. 그러면 Write가 차단됩니다. Claude가 "개인정보를 포함한 파일을 저장해 줘" 라는 지시를 받아도, Hook이 막아 줍니다.

효과

- **컴플라이언스 안심**: Claude의 판단에 의존하지 않고 기계적으로 차단합니다.
- **다양한 활용**: 코드 포맷팅, 테스트 자동 실행, 변경 로그 기록 등 다양한 용도로 활용됩니다.
- **타이밍 선택**: PreToolUse(실행 전), PostToolUse(실행 후), Stop(작업 종료 시) 등을 선택할 수 있습니다.

2.9 컨텍스트 관리

컨텍스트란

Claude가 한 번에 "기억"할 수 있는 대화 분량입니다. 너무 길어지면 처음 부분이 잊혀지거나 응답 속도가 느려집니다. 그래서 컨텍스트를 적절히 관리하는 것이 중요합니다.

핵심 명령어

명령	효과
`/clear`	현재 대화 완전 초기화 (새 세션 시작)
`/compact`	대화를 요약해서 토큰 절약 (흐름 유지)
`/memory`	영구 메모리 항목 관리
`/context`	현재 컨텍스트 사용량 확인

노트 테이킹

긴 작업 중간에 핵심을 메모해 두면 `/clear` 후에도 흐름을 이어 갈 수 있습니다.

```
> 이 프로젝트는 AI 추진 TF의 "AI 추진 계획 보고서 자동화" 야.
> 메모리에 기억해 줘.
```

Claude는 이 정보를 영구 메모리(`~/.claude.json`)에 저장합니다. 다음 세션에서 `/clear` 한 뒤에도 "이 프로젝트가 뭐였지?" 라고 물으면 정확히 답합니다.

2.10 도구의 병렬 실행

Claude Code의 핵심 도구

도구	용도
Read	파일 읽기
Write	파일 쓰기
Edit	파일 수정
Bash	셸 명령 실행
WebFetch	웹 콘텐츠 가져오기

이 도구들은 **동시에 여러 개 호출**할 수 있습니다. 예를 들어 "보고서와 PPT를 동시에 만들어 줘" 라고 하면, 두 작업이 병렬로 진행되어 시간이 절반으로 줄어듭니다.

병렬 실행을 적극적으로 활용하면 같은 시간에 더 많은 작업을 끝낼 수 있습니다. 단, 작업끼리 의존성이 있으면(앞 결과가 뒤의 입력이면) 자동으로 순차 실행됩니다.

2.11 성공 패턴 — 빠른 반복이 완벽을 이깁니다

처음부터 완벽한 결과를 얻으려고 하시면 안 됩니다. AI 코딩의 핵심은 **빠른 반복**입니다.

1. 일단 만들어 봅니다 (60점이어도 괜찮습니다).
2. 결과를 보고 수정 요청합니다 ("표를 더 구체적으로", "이 섹션은 빼 줘").
3. 다시 만듭니다 (75점).
4. 만족할 때까지 반복합니다.

3-4회 반복하면 처음부터 완벽하게 만들려고 한 것보다 더 좋은 결과가 나옵니다. AI에게 한 번에 완벽한 지시를 내리는 것보다, 실수해도 빠르게 고치는 게 효율적입니다. 이것이 바이브코딩의 핵심 정신입니다.

2.12 바이브코딩 주의사항

위험	방지법
할루시네이션 (사실과 다른 답)	출처 요구, 결과 직접 검증, 중요한 수치는 원본 확인
보안 위험	Hook으로 개인정보·키 차단, 권한 좁게 설정
저작권	외부 데이터 사용 시 출처 명시, 허가된 자료만 사용
잘못된 자동화	항상 결과 검토 후 사용, 처음 몇 번은 사람이 중간 체크
비밀 유출	API 키, 토큰을 코드에 하드코딩하지 말고 환경 변수 사용

AI가 아무리 똑똑해도 결정의 책임은 사람에게 있습니다. AI는 보조 도구이지, 의사결정자가 아닙니다.

Part 3. 기능 체험

이 파트에서는 Part 2에서 배운 개념을 직접 손으로 만들어 봅니다. 모든 미션은 **공통 주제**로 진행합니다: "AI 추진 계획 보고서 자동화 — AI 추진 TF". 옆 사람과 결과물을 비교해 보시면, 같은 주제와 같은 도구로 거의 비슷한 결과가 나오는 것을 확인할 수 있습니다.

각 미션은 다음과 같이 구성되어 있습니다.

- ⌧**GOAL** — 이 미션에서 배우는 것
- ⌧**INPUT** — 무엇을 입력하는지
- ⌧**OUTPUT** — 어떤 파일이 나와야 하는지 (파일 칩으로 시각화)
- **프롬프트** — 그대로 복사하거나 직접 작성
- ⌧**필수 체크리스트** — 통과해야 미션 클리어
- ⌧**도전 체크리스트** — 선택, 더 좋은 결과물을 위한 추가 기준
- ⌧**AI 채점** — 산출물을 AI가 읽고 항목별 ✓/✗ + 점수 + 코멘트 (파일 생성 후 활성화)

미션 1. Plan 모드로 기능 정의하기

```
/plan AI 추진 TF의 "AI 추진 계획 보고서"를 자동으로 만드는
프로그램을 설계할 거야. 단계별 계획을 세우고
PLAN.md 파일로 저장해줘.
```

목표 파일: `PLAN.md`

기대 결과: 4-5단계 보고서 생성 워크플로우 (배경 → 사례 → 로드맵 → 위험·기대효과 → 출력)가 PLAN.md에 정리되어 있습니다.

미션 2. 권한 설정하기

```
.claude/settings.local.json 파일을 만들어서
Bash, Read, Write, Edit 도구를 자동 허용 설정해줘.
AI 추진 계획 보고서 자동화에 필요한 권한이야.
```

목표 파일: `.claude/settings.local.json`

기대 결과: permissions.allow 에 Bash, Read, Write, Edit 추가. 다음 작업부터 Yes를 누르지 않아도 됩니다.

미션 3. CLAUDE.md 만들기

```
현재 폴더에 새 CLAUDE.md를 만들어줘.
팀은 "AI 추진 TF", 프로젝트는 "AI 추진 계획 보고서 자동화".
한국어로 작성, 브랜드 색상 #F58220 명시, 250단어 이내.
```

목표 파일: `CLAUDE.md` (한국어 + #F58220 포함)

기대 결과: 프로젝트 루트에 CLAUDE.md 생성. 팀 정보, 코딩 규칙, 주요 작업 절차가 기록되어 있습니다.

미션 4. Skill 만들기

```
AI 추진 계획 보고서 작성 표준 절차를 스킬로 만들어줘.
.claude/skills/ai-plan-report/SKILL.md 경로에 작성.
입력, 절차(5단계), 출력 형식을 명시. 250단어 이내.
```

목표 파일: `.claude/skills/ai-plan-report/SKILL.md`

기대 결과: SKILL.md 파일에 입력·절차·출력 명시. 다음에 "AI 추진 보고서 만들어 줘" 한 줄로 이 절차를 호출할 수 있습니다.

미션 5. Command 만들기

```
"/ai-plan" 슬래시 커맨드를 .claude/commands/ai-plan.md 로 만들어줘.
인자로 부서명을 받아 ai-plan-report 스킬을 호출해서
보고서를 만드는 워크플로우야. 사용 예: /ai-plan 영업기획팀
```

목표 파일: `.claude/commands/ai-plan.md`

기대 결과: `/ai-plan 영업기획팀` 한 줄로 전체 워크플로우 실행. 부서별 맞춤 보고서가 outputs/ 에 저장됩니다.

미션 6. Hook 만들기

```
보고서를 저장하기 전에 개인정보(주민번호, 전화번호, 이메일)가
있으면 차단하는 훅 만들어줘. .claude/hooks/check-pii.sh
스크립트와 settings.local.json 의 PreToolUse hook 등록.
```

목표 파일: `.claude/hooks/check-pii.sh`

기대 결과: Write 호출 시 PII 패턴이 감지되면 거부됩니다. 가짜 주민번호로 테스트하면 차단되는 것을 확인할 수 있습니다.

미션 7. 컨텍스트 관리

```
지금까지 AI 추진 계획 보고서 프로젝트로 진행한 사항들을
기억해줘. 그 다음 /compact 실행해줘.
```

기대 결과: 대화 요약 + 컨텍스트 압축. 압축 후에도 다음 질문에서 컨텍스트가 유지됩니다.

미션 8. 노트 테이킹

```
이 프로젝트는 AI 추진 TF의 "AI 추진 계획 보고서 자동화" 야.
이 정보를 메모리에 기억해줘. 다음에 물어보면 알려줘야 해.
```

기대 결과: `~/.claude.json` 메모리에 영구 저장. `/clear` 후 새 대화에서 "이 프로젝트가 뭐였지?" 라고 물으면 정확히 답합니다.

Part 4. 실습 프로젝트

지금까지 배운 모든 도구를 한 번에 활용해서, 본인 업무에 맞는 자동화를 처음부터 끝까지 만듭니다. 그리고 마지막에는 이 자동화를 다른 사람도 쓸 수 있도록 웹페이지로 확장합니다.

4.1 7단계 워크플로우

작업 위치: `~/subagent-mastery` (이미 준비되어 있습니다)

1. **Step 1. Plan** — 만들 프로그램의 단계별 설계 (PLAN.md 저장)
2. **Step 2. CLAUDE.md** — 본인 업무에 맞는 프로젝트 규칙서 작성
3. **Step 3. Skill** — 핵심 업무 절차를 스킬로 정리
4. **Step 4. Command** — 자주 쓸 워크플로우를 단축 명령어로
5. **Step 5. Hook** — 안전장치 추가
6. **Step 6. 실행** — 자기 부서 주제로 직접 돌려 보기
7. **Step 7. 웹페이지** — 완성된 기능을 HTML로 감싸 다른 사람도 쓸 수 있게

각 Step의 프롬프트는 슬라이드에 그대로 들어 있고, 자동검증이 산출물 파일을 확인합니다. 막히면 강사에게 손 들어 주세요.

4.2 완성하면 무엇이 손에 남을까요

워크숍이 끝나면 여러분 손에는 다음이 남습니다.

- **Plan 모드** — 설계 → 검토 → 승인 단계
- **CLAUDE.md** — 매번 자동 적용되는 프로젝트 규칙
- **Skill** — 재사용 가능한 표준 업무 매뉴얼
- **Command** — `/명령어` 한 줄로 전체 워크플로우 호출
- **Hook** — 예외 없는 자동 안전장치
- **Web UI** — 다른 사람도 쓸 수 있는 HTML 인터페이스

이 여섯 가지 조합이 여러분의 첫 "AI 자동화 도구"입니다. 다음에 새로운 자동화가 필요할 때 같은 패턴을 반복하시면 됩니다.

Appendix A. 워크숍 이후 — 집에서 혼자 쓰기

워크숍 환경은 5시간 동안만 유지됩니다. 워크숍이 끝난 뒤에 본인 PC에서 직접 사용하고 싶으시다면, 이 부록을 따라 환경을 갖추시면 됩니다.

Claude를 본인 PC에서 쓰는 방법은 크게 두 가지입니다.

1. **Claude Code (CLI)** — 워크숍에서 사용한 터미널 기반 도구. 파일 자동 생성·코드 실행 가능.
2. **Claude Desktop (GUI 앱)** — 채팅창 형태의 데스크톱 앱. 가볍고 직관적이지만 파일 작업은 제한적.

처음 쓰시는 분께는 **둘 다 설치해 두는 것**을 권장합니다. 가볍게 물어볼 때는 Desktop, 본격적으로 자동화할 때는 Code를 쓰시면 됩니다.

A.1 Claude Code (CLI) 개인 PC 설치

사전 준비

- **Node.js 18 이상** — `node --version` 으로 확인
- **Git** (Windows의 경우 Git for Windows에 포함된 Git Bash)
- **Anthropic 계정** — claude.ai 에서 가입

설치 (모든 OS 공통)

```
npm install -g @anthropic-ai/claude-code
```

설치가 끝나면 터미널 어디에서든 `claude` 한 줄로 시작할 수 있습니다.

첫 실행

```
cd ~/내-프로젝트-폴더
claude
```

처음 실행하면 Claude가 로그인 절차를 안내합니다.

```
> /login
```

표시되는 URL을 본인 PC 브라우저에서 열어 Claude 계정으로 로그인하시고, 받은 코드를 터미널에 붙여넣으시면 인증이 완료됩니다. 한 번 인증하면 다음부터는 자동으로 유지됩니다.

Windows 사용자 주의

- Claude Code는 내부적으로 bash 셸을 사용하므로 **Git Bash 또는 WSL**이 필요합니다.
- Git for Windows를 설치하면 Git Bash가 함께 설치됩니다.
- PowerShell이나 cmd에서도 동작하지만 Git Bash 환경이 가장 안정적입니다.

A.2 Claude Desktop (GUI 앱) 설치

다운로드

claude.ai 의 다운로드 페이지에서 본인 OS에 맞는 설치 파일을 받으십시오.

- **Windows**: `Claude-Setup.exe`
- **macOS**: `Claude.dmg`
- **Linux**: 공식 .deb 또는 AppImage 제공

설치는 일반 앱 설치와 동일합니다. 다음, 다음을 누르시면 끝납니다.

첫 실행

앱을 켜고 Claude 계정으로 로그인하시면 채팅 화면이 나타납니다. 바로 질문을 입력하시면 됩니다.

```
오늘 해야 할 일을 정리해 줘
```

Desktop은 기본적으로 ChatGPT와 비슷한 채팅 경험을 제공합니다. 단, Anthropic 계정 한 개로 Desktop과 Code 양쪽에서 모두 사용할 수 있어 사용량이 합산됩니다.

MCP 연결 (선택)

Desktop에서도 외부 도구를 연결할 수 있습니다. 메뉴 → 설정 → MCP 항목에서 Filesystem, GitHub, Slack 등의 커넥터를 켤 수 있습니다. 처음에는 설정 없이 그대로 쓰셔도 충분합니다.

A.3 CLI vs Desktop 비교

구분	Claude Code (CLI)	Claude Desktop (GUI)
인터페이스	터미널 (검은 창)	채팅 앱 (윈도우/맥 GUI)
학습 곡선	약간 있음 (터미널 익숙해야)	매우 쉬움 (카톡 수준)
파일 생성·수정	자유롭게 가능	제한적 (대화 첨부만)
코드 직접 실행	가능 (Bash, Python 등)	불가
폴더 단위 작업	매우 강력 (전체 프로젝트 분석)	단일 첨부 위주
CLAUDE.md / Skill / Hook	모두 사용 가능	사용 불가
슬래시 커맨드	본인이 직접 만들 수 있음	기본 제공 명령만
자동화 워크플로우	핵심 기능	거의 불가
적합한 용도	보고서 자동화, 분석, 코드 작성	일상 질문, 글쓰기, 아이디어
비용 모델	Pro/Max 구독 동일	Pro/Max 구독 동일

한 줄 요약

- 가벼운 질문, 글쓰기, 아이디어 정리 → **Desktop**
- 보고서 자동화, 파일 작업, 반복 업무 → **Code**
- 처음에는 둘 다 설치해 두고 상황에 맞게 골라 쓰시는 것이 가장 효율적입니다.

A.4 실행 흐름과 자주 쓰는 명령

Code의 일반적인 작업 흐름

1. 작업 폴더로 이동: `cd ~/projects/my-task`
2. Claude Code 실행: `claude`
3. 자연어로 지시: `> 이 폴더의 *.csv를 합쳐서 summary.xlsx 만들어줘`
4. Claude가 작업 → 결과 파일 폴더에 생성
5. 필요시 수정 요청: `> 매출 컬럼은 천 단위 콤마 추가해줘`
6. 종료: `Ctrl+C` 두 번 또는 `> /exit`

자주 쓰는 명령어

명령	효과
`/login`	Claude 계정 로그인
`/logout`	로그아웃
`/clear`	현재 대화 초기화
`/compact`	대화 요약 (긴 대화 시)
`/memory`	영구 메모리 관리
`/context`	컨텍스트 사용량 확인
`/usage`	Claude Max 사용량 확인
`/help`	명령어 목록
`/exit`	종료

A.5 안전하게 쓰기 위한 주의사항

권한과 폴더

- **작업 폴더 분리**: 회사 자료, 개인 자료를 폴더 단위로 분리하시고, Claude Code는 그 폴더 안에서만 실행하십시오. (Claude Code는 실행한 폴더만 인식합니다.)
- **권한 좁히기**: 처음에는 `Bash(*)` 같은 와일드카드 권한을 피하고, `Bash(python *)` 처럼 좁은 패턴부터 시작하십시오.
- **CLAUDE.md에 보안 규칙**: "이 프로젝트에서는 ~/Documents 밖으로 절대 파일을 쓰지 말 것" 같은 규칙을 명시해 두면 큰 사고를 막을 수 있습니다.

위험한 명령

다음 명령은 절대 자동 허용하지 마십시오. 매번 직접 확인하시는 것이 안전합니다.

- `rm -rf` (파일 영구 삭제)
- `git push --force` (팀원의 코드 덮어쓰기)
- `sudo` (관리자 권한)
- 외부 서버 호출 (`curl` , `wget`)

데이터와 개인정보

- 회사 내부 자료를 Claude에게 줄 때는 회사 정책을 먼저 확인하십시오. Anthropic은 기본적으로 사용자 입력을 학습에 쓰지 않지만, 정책 변경 가능성이 있으니 가이드를 확인하시는 것이 좋습니다.
- 주민번호, 전화번호, 이메일 같은 개인정보는 Hook으로 자동 차단하시는 것을 권장합니다 (2.8 절 참고).
- 파일을 삭제하거나 덮어쓰는 작업은 Claude에게 시키기 전에 백업을 만드시는 것이 좋습니다.

비용 관리

- Claude Pro/Max 구독을 사용하시면 월 정액으로 충분히 활용할 수 있습니다.
- API 키 직접 사용은 토큰 단위 과금이라 주의가 필요합니다 (대규모 작업 시 비용 빠르게 증가).
- 본인 사용량은 Code/Desktop 양쪽에서 `/usage` 명령으로 확인할 수 있습니다.

결과물 검증

- AI가 만든 보고서는 **반드시 사람이 검토** 후 사용하십시오. 특히 외부에 나가는 문서, 의사결정 자료는 사실 확인이 필수입니다.
- 출처 각주가 있는지, 수치가 정확한지, 톤이 적절한지 항상 한 번 더 보십시오.
- "AI가 만들었으니 맞을 것" 이라는 가정은 위험합니다.

미래에셋생명 · AI 코딩 어시스턴트 워크숍 Claude Code · 2026